Lab 4: Node, Express, Handlebars, SQLite3

This lab session is dedicated to using Node, Express, Handlebars, and SQLite3 in your project. During the labs, a "projects and skills" database will be used. To try and test some code, you can use the data I provide for the labs, but in the end, the database of your project MUST use **YOUR (OWN) DATA**!

1) Creating the tables and inserting data in them with Javascript

During lab 3, you created your project directory and populated it with Node, Express, and SQLite3. We will start with your existing code and adapt it to use SQLite3 in our project. Add the skills JSON variable to your code (you can get it from the "model-lab4.js" file on Canvas).

```
// MODEL for skills
const skills=[
    {"id":"1", "name":"PHP", "type":"Programming language", "desc":"Programming with PHP on the server side.", "level":4},
    {"id":"2", "name":"Python", "type":"Programming language", "desc":"Programming with Python.", "level":4},
    {"id":"3", "name":"Java", "type":"Programming language", "desc":"Programming with Java.", "level":2},
    {"id":"4", "name":"ImageJ", "type":"Framework", "desc":"Java Framework for Image Processing.", "level":2},
    {"id":"5", "name":"Javascript", "type":"Programming language", "desc":"Programming with Javascript on the client side.",
    "level":4},
    {"id":"6", "name":"Node", "type":"Programming language", "desc":"Programming with Javascript on the server side.",
    "level":4},
    {"id":"7", "name":"Express", "type":"Framework", "desc":"A framework for programming Javascript on the server side.",
    "level":4},
    {"id":"8", "name":"Scikit-image", "type":"Library", "desc":"A library for Image Processing with Python.", "level":3},
    {"id":"9", "name":"OpenCV", "type":"Library", "desc":"A library for Image Processing with Python.", "level":4},
    {"id":"10", "name":"LaTeX", "type":"Description language", "desc":"A language to describe and build professional
    documents.", "level":5},
    {"id":"11", "name":"HTML", "type":"Description language", "desc":"A language to create web pages.", "level":4},
    {"id":"12", "name":"CSS", "type":"Description language", "desc":"A language to apply styles to web pages.", "level":4},
    {"id":"13", "name":"C", "type":"Programming language", "desc":"The historical language of the Linux/Unix kernels.",
    "level":3},
    {"id":"14", "name":"C++", "type":"Programming language", "desc":"A fast high level programming language.", "level":1},
    {"id":"15", "name":"SQL", "type":"Query language", "desc":"The relational database language to access data (CRUD).",
    "level":4},
]
```

Add the code to create the "skills" table to your database:

```
// create table skills at startup
db.run("CREATE TABLE skills (sid INTEGER PRIMARY KEY AUTOINCREMENT, sname TEXT NOT NULL, sdesc TEXT NOT NULL, stype TEXT NOT NULL, slevel INT)", (error) => {
    if (error) {
        console.log("ERROR: ", error) // error: display it in the terminal
    } else {
        console.log("---> Table projects created!")       // no error, the table has been created

        // inserts skills
        skills.forEach( (oneSkill) => {
            db.run("INSERT INTO skills (sid, sname, sdesc, stype, slevel) VALUES (?, ?, ?, ?, ?)",
                [oneSkill.id, oneSkill.name, oneSkill.desc, oneSkill.type, oneSkill.level], (error) => {
                if (error) {
                    console.log("ERROR: ", error)
                } else {
                    console.log("Line added into the skills table!")
                }
            })
        })
    }
})
```

Do the same for the "projects" table using the "projects" JSON variable (the one used in lab 3). Or, even better, do it for YOUR (OWN) DATA in your project! :)

You must install the "sqlite3" package in your project directory to be able to use it in your code:

```
const sqlite3=require('sqlite3') // load the sqlite3 package
```

```
//---------
// DATABASE
//---------
const dbFile='my-project-data.sqlite3.db'
db=new sqlite3.Database(dbFile)
```

2) Creating functions

To get a simpler organization of our code, we can create **functions**!

Source: (ChatGPT, 2024): “a **function** is a reusable block of code designed to perform a specific task or computation. It typically takes some input (called **parameters** or **arguments**), processes it, and returns an output (though not all functions return a value).”

So, let’s create a function initTableSkills() that will create the table “skills” and populate it at the start of the server (i.e. in the listen function!).

```
//---------------
// USER FUNCTIONS
//---------------

function initTableSkills(mydb) {
    // MODEL for skills
    const skills=[
        {"id":"1", "name":"PHP", "type":"Programming language", "desc":"Programming with PHP on the server side.", "level":4},
        {"id":"2", "name":"Python", "type":"Programming language", "desc":"Programming with Python.", "level":4},
        {"id":"3", "name":"Java", "type":"Programming language", "desc":"Programming with Java.", "level":2},
        {"id":"4", "name":"ImageJ", "type":"Framework", "desc":"Java Framework for Image Processing.", "level":2},
        {"id":"5", "name":"Javascript", "type":"Programming language", "desc":"Programming with Javascript on the client side.", "level":4},
        {"id":"6", "name":"Node", "type":"Programming language", "desc":"Programming with Javascript on the server side.", "level":4},
        {"id":"7", "name":"Express", "type":"Framework", "desc":"A framework for programming Javascript on the server side.", "level":4},
        {"id":"8", "name":"Scikit-image", "type":"Library", "desc":"A library for Image Processing with Python.", "level":3},
        {"id":"9", "name":"OpenCV", "type":"Library", "desc":"A library for Image Processing with Python.", "level":4},
        {"id":"10", "name":"LaTeX", "type":"Description language", "desc":"A language to describe and build professional documents.", "level":5},
        {"id":"11", "name":"HTML", "type":"Description language", "desc":"A language to create web pages.", "level":4},
        {"id":"12", "name":"CSS", "type":"Description language", "desc":"A language to apply styles to web pages.", "level":4},
        {"id":"13", "name":"C", "type":"Programming language", "desc":"The historical language of the Linux/Unix kernels.", "level":3},
        {"id":"14", "name":"C++", "type":"Programming language", "desc":"A fast high level programming language.", "level":1},
        {"id":"15", "name":"SQL", "type":"Query language", "desc":"The relational database language to access data (CRUD).", "level":4},
    ]

    // create table skills at startup
    mydb.run("CREATE TABLE skills (sid INTEGER PRIMARY KEY AUTOINCREMENT, sname TEXT NOT NULL, sdesc TEXT NOT NULL, stype TEXT NOT NULL, slevel INT)", (error) => {
        if (error) {
            console.log("ERROR: ", error) // error: display it in the terminal
        } else {
            console.log("---> Table projects created!")       // no error, the table has been created

            // inserts skills
            skills.forEach( (oneSkill) => {
                db.run("INSERT INTO skills (sid, sname, sdesc, stype, slevel) VALUES (?, ?, ?, ?, ?)",
                    [oneSkill.id, oneSkill.name, oneSkill.desc, oneSkill.type, oneSkill.level], (error) => {
                    if (error) {
                        console.log("ERROR: ", error)
                    } else {
                        console.log("Line added into the skills table!")
                    }
                })
            })
        }
    })
}
```

Then, we must change the listen function code to run this function at startup:

```
//-------
// LISTEN
//-------
app.listen(port, function () { // listen to the port
    initTableSkills(db) // create the table skills and populate it
    // displays a message in the terminal when the server is listening
    console.log('Server is listening on port '+port+'...')
})
```

Please remove any other code used to create and populate your tables. They should only be in your initTableBlabla() functions! (where “Blabla” is the name of your tables).

You must create a function for the second table. In my case, it is the table “projects”.

```
function initTableProjects(mydb) {
    // MODEL for projects
    const projects=[
        { "id":"1", "name":"Counting people with a camera", "type":"Research", "desc":"The purpose of this project is to count people passing through a corridor
        and to know how many are in the room at a certain time.", "year":2022, "dev":"Python and OpenCV (Computer vision) library", "url":"/img/counting.png" },
        { "id":"2", "name":"Visualisation of 3D medical images", "type":"Research", "desc":"The project makes a 3D model of the analysis of the body of a person
        and displays the detected health problems. It is useful for doctors to view in 3D their patients and the evolution of a disease.", "year":2012, "url":"/
        img/medical.png" },
        { "id":"3", "name":"Multiple questions system", "type":"Teaching", "desc":"During the lockdowns in France, this project was useful to test the students
        online with a Quizz system.", "year":2021, "url":"/img/qcm07.png" },
        { "id":"4", "name":"Image comparison with the Local Dissmilarity Map", "desc":"The project is about finding and quantifying the differences between two
        images of the same size. The applications were numerous: satellite imaging, medical imaging,...", "year":2020, "type":"Research", "url":"/img/diaw02.
        png" },
        { "id":"5", "name":"Management system for students' internships", "desc":"This project was about the creation of a database to manage the students'
        internships.", "year":2012, "type":"Teaching", "url":"/img/management.png" },
        { "id":"6", "name":"Magnetic Resonance Spectroscopy", "desc":"Analysis of signals and images from Magnetic Resonance Spectroscopy and Imaging.",
        "year":2013, "type":"Research", "url":"/img/yu00.png" },
        { "id":"7", "name":"Signal Analysis for Detection of Epileptic Deseases", "desc":"This project was about the detection of epileptic problems in signals.
        ", "year":2019, "type":"Research", "url":"/img/youssef00.png" },
    ]

    // create table projects at startup
    mydb.run("CREATE TABLE projects (pid INTEGER PRIMARY KEY AUTOINCREMENT, pname TEXT NOT NULL, pyear INTEGER NOT NULL, pdesc TEXT NOT NULL, ptype TEXT NOT
    NULL, pimgURL TEXT NOT NULL)", (error) => {
        if (error) {
            console.log("ERROR: ", error) // error: display it in the terminal
        } else {
            console.log("---> Table projects created!")       // no error, the table has been created

            // inserts projects
            projects.forEach( (oneProject) => {
                db.run("INSERT INTO projects (pid, pname, pyear, pdesc, ptype, pimgURL) VALUES (?, ?, ?, ?, ?, ?)", [oneProject.id, oneProject.name, oneProject.
                year, oneProject.desc, oneProject.type, oneProject.url], (error) => {
                    if (error) {
                        console.log("ERROR: ", error)
                    } else {
                        console.log("Line added into the projects table!")
                    }
                })
            })
        }
    })
}
```

When you run this code, if your tables already exist, you will get an error message. You can add comments in front of the lines in your listen() function to avoid this error messages.

```
PS C:\Users\lanjer\Desktop\project-2024-wdf-lab4> node .\server.js
Server is listening on port 8080...
ERROR:  [Error: SQLITE_ERROR: table skills already exists] {
  errno: 1,
  code: 'SQLITE_ERROR'
}
ERROR:  [Error: SQLITE_ERROR: table projects already exists] {
  errno: 1,
  code: 'SQLITE_ERROR'
}
```

```
//-------
// LISTEN
//-------
app.listen(port, function () { // listen to the port
    //initTableSkills(db) // create the table skills and populate it
    //initTableProjects(db) // create the table projects and populate it
    // displays a message in the terminal when the server is listening
    console.log('Server is listening on port '+port+'...')
})
```

```
PS C:\Users\lanjer\Desktop\project-2024-wdf-lab4> node .\server.js
Server is listening on port 8080...
```

3) Getting data from the tables

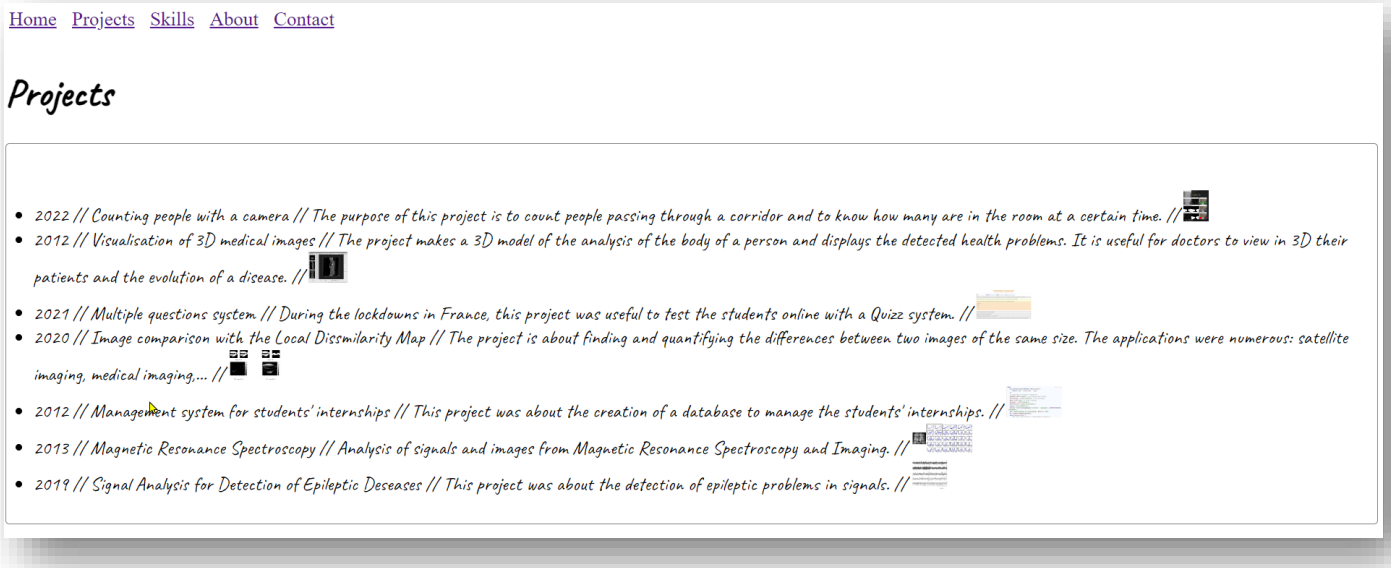
In lab 3, the data displayed on the “views/projects.handlebars” file came from a JSON variable. Now, for the project, it **MUST come from the database**. It means that we need to build a model with data and send it to the view when we call the “render()” function.

```
// create a new route to send back the projects page
app.get('/skills', function (req, res) {
    db.all("SELECT * FROM skills", (error, listOfSkills) => {
        if (error) {
            console.log("ERROR: ", error) // error: display in terminal
        } else {
            model={ skills: listOfSkills }
            res.render('skills.handlebars', model)
        }
    })
})
```

views > skills.handlebars > ...

```
1   <div id="principal">
2       <h1>Skills</h1>
3       <div class="mytextcard">
4           <ul>
5               {{#each skills}}
6                   <li>
7                       {{sname}} - {{stype}} - {{sdesc}}
8                   </li>
9               {{/each}}
10          </ul>
11      </div>
12  </div>
```

Do the same thing for the “/projects” route.

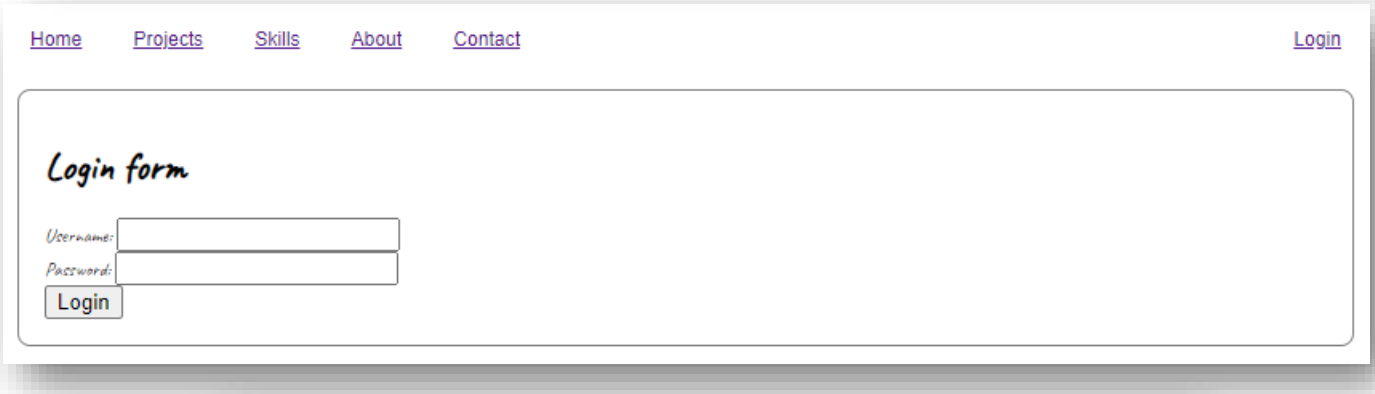


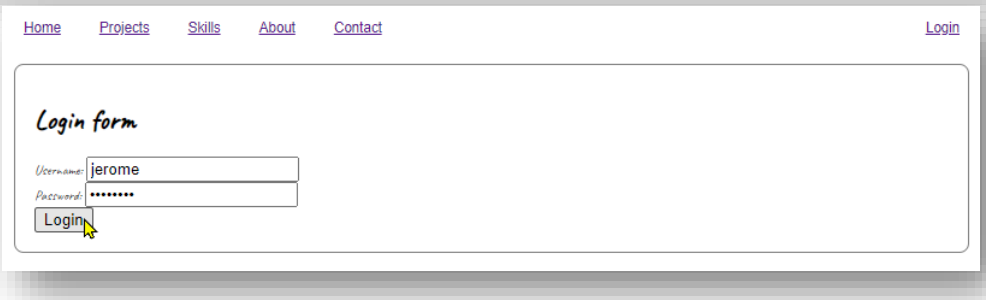
4) Get ready for the login part

Please, add a “Login” element in the menu and a “/login” route in your code. This route must lead to a “login.handlebars” page. This page must contain a form with login and password and a submit button.

```
app.get('/login', (req, res) => {
    res.render('login.handlebars')
})
```

```
<div id="principal">
    <div class="mytextcard">
        <h1>Login form</h1>
        <form action="/login" method="post">
            <label for="username">Username:</label> <input type="text" id="username" name="username"><br />
            <label for="password">Password:</label> <input type="password" id="password" name="password"><br />
            <input type="submit" value="Login">
        </form>
    </div>
</div>
```





To use the POST method in the form and be able to get the data from any form, we will use the “urlencoded” middleware:

```
//------------
// MIDDLEWARES
//------------
// middleware to search for unknow routes into the 'public' directory
app.use(express.static('public')) // the public directory is static
// using express middleware for processing forms sent using the "post" method
app.use(express.urlencoded({ extended: true }));
```

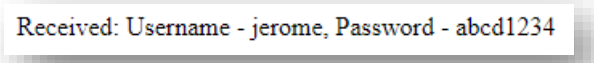
```
app.post('/login', (req, res) => {

    // Alternative 1
    const { username, password } = req.body

    // Alternative 2
    //const username = req.body.username;
    //const password = req.body.password;

    // verification steps
    if (!username || !password) {
        return res.status(400).send('Username and password are required.');
    }
    // sending a response
    res.send(`Received: Username - ${username}, Password - ${password}`);
})
```

Test your login and check if you can send the data from the form to your browser:



Please, remember:

Web Dev is **Fun**

:)